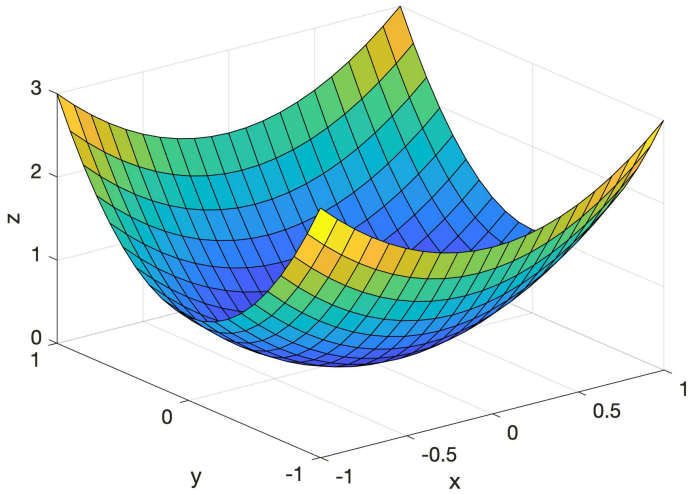
Dynamic Convergence of Multivariate Functions

Javier Pérez Calderón

Introduction



Theoretical background

- First-order
  - Gradient Descent (+ Nesterov momentum): O(n)
  - Newton: O(n^3)
    - Pure Newton
    - Damped Newton (Armijo condition)
  - BFGS: O(n^2)
  - L-BFGS: O(mn)

Cost per iteration ⇔ Number of iterations

Methodology

1. Damped Newton: Quadratical (en+1 <= c en^2) <= c^(n+1) e0
2. BFGS: Superlinear (n° of iterations: geometric mean of Damped Newton and Gradient Descent)
3. L-BFGS: Superlinear (n° of iterations: geometric mean of BFGS and Gradient Descent)
4. Gradient Descent: Linear (en+1 = c en)

Error = ||Grad f(x)||

Estimated time = n° iterations * time per iteration

Methodology

- f(x1, x2) = x1 cos(x1) + x2 sin(x2)
- f(x1,...,xn) = x1^2 + ... + xn^2
- f(x1,...,xn) = x1exp(x1) + ... + xnexp(xn)
- f(x1,x2) = (a-x1)^2 + b(x2-x1^2)^2 (Rosenbrock function)

Methodology

```python
import time
import numpy as np

def damped_newton(f, grad_f, hessian_f, x0, max_duration = None, max_steps = None, alpha0: float = 1.0, c1: float = 1e-4,
                  rho: float = 0.5, epsilon: float = 1e-4):
    if (max_duration is None and max_steps is None):
        raise ValueError("You have to set a maximum duration or maximum number of steps to avoid infinite loops")

    x = x0
    steps = 0
    start = time.time()
    while True:
        pk = np.linalg.solve(hessian_f(x), -grad_f(x))
        alpha = alpha0
        while (f(x+alpha*pk)> f(x) + c1*alpha * grad_f(x).T @ pk):
            alpha *= rho
        x += alpha*pk
        steps += 1

        if np.linalg.norm(grad_f(x)) <= epsilon:
            break
        if max_duration is not None and (time.time() - start >= max_duration):
            break
        if max_steps is not None and steps == max_steps:
            break
    return x
```

Methodology

```python
import time
import numpy as np

def bfgs(f, grad_f, x0, max_duration = None, max_steps = None, alpha0: float = 1.0, c1: float = 1e-4, rho: float = 0.5,
         epsilon: float = 1e-4):
    if (max_duration is None and max_steps is None):
        raise ValueError("You have to set a maximum duration or maximum number of steps to avoid infinite loops")

    B = np.eye(len(grad_f(x0)))
    xk = x0
    steps = 0
    start = time.time()
    while True:
        pk = np.linalg.solve(B, -grad_f(xk))
        alpha = alpha0
        # Armijo and curvature conditions
        while (f(xk + alpha*pk) > f(xk) + c1*alpha * grad_f(xk).T @ pk):
            alpha *= rho
        xk_1 = xk + alpha*pk
        steps += 1

        sk = xk_1 - xk
        yk = grad_f(xk_1) - grad_f(xk)
        if(yk.T@sk > 0):
            B += - (B @ sk @ sk.T @ B)/(sk.T @ B @ sk) + (yk @ yk.T)/(yk.T @ sk)
        xk = xk_1

        if np.linalg.norm(grad_f(xk_1)) <= epsilon:
            break
        if max_duration is not None and (time.time() - start >= max_duration):
            break
        if max_steps is not None and steps == max_steps:
            break

    return xk
```

```python
import time
import numpy as np
from collections import deque

def l_bfgs(f, grad_f, x0, max_duration = None, max_steps = None, m: int = 5, alpha0: float = 1.0, c1: float = 1e-4,
           rho: float = 0.5, epsilon: float = 1e-4):
    if (max_duration is None and max_steps is None):
        raise ValueError("You have to set a maximum duration or maximum number of steps to avoid infinite loops")

    xk = x0
    memory = deque(maxlen=m)
    steps = 0
    start = time.time()

    while True:
        q = grad_f(xk)

        for i in range(len(memory)):
            alphai = memory[-i-1]["rhok"] * (memory[-i-1]["sk"].T @ q)
            q -= alphai * memory[-i-1]["yk"]
            memory[-i-1]["alpha"] = alphai

        if steps == 0:
            r = q
        else:
            phik = (memory[-1]["yk"].T @ memory[-1]["sk"]) / (memory[-1]["yk"].T @ memory[-1]["yk"])
            r = phik * q

        for i in range(len(memory)):
            beta = memory[i]["rhok"] * (memory[i]["yk"].T @ r)
            r += memory[i]["sk"]*(memory[i]["alpha"] - beta)

        pk = -r

        alpha = alpha0
        while (f(xk + alpha*pk) > f(xk) + c1*alpha * grad_f(xk).T @ pk):
            alpha *= rho
        xk_1 = xk + alpha*pk
        steps += 1

        sk = xk_1 - xk
        yk = grad_f(xk_1) - grad_f(xk)
        xk = xk_1

        if np.linalg.norm(grad_f(xk_1)) <= epsilon:
            break
        if max_duration is not None and (time.time() - start >= max_duration):
            break
        if max_steps is not None and steps == max_steps:
            break

        if(yk.T@sk > 0):
            rhok = 1 / (yk.T @ sk)
            if len(memory) == m:
                memory.popleft()
            memory.append({"sk": sk, "yk": yk, "rhok": rhok})

    return xk
```

Methodology

```python
import time
import numpy as np

def gradient_descent(grad_f, x0, max_duration = None, max_steps = None, lr: float = 0.01, beta: float = 0.9, epsilon: float = 1e-4):
    if (max_duration is None and max_steps is None):
        raise ValueError("You have to set a maximum duration or maximum number of steps to avoid infinite loops")

    x = x0
    momentum = np.zeros(grad_f(x).shape)
    steps = 0
    start = time.time()
    while True:
        x_tilde = x + beta*momentum
        new_momentum = -lr * grad_f(x_tilde) + beta*momentum
        x += new_momentum
        momentum = new_momentum
        steps += 1

        if (np.linalg.norm(grad_f(x)) <= epsilon):
            break
        if max_duration is not None and (time.time() - start >= max_duration):
            break
        if max_steps is not None and steps == max_steps:
            break
    return x
```

Methodology

```python
from gradient_descent import gradient_descent
from damped_newton import damped_newton
from bfgs import bfgs
from l_bfgs import l_bfgs
import time
import math
import numpy as np

class Descent:
    def __init__(self, f, grad_f, hessian_f, epsilon: float = 1e-4):
        self.f = f
        self.grad_f = grad_f
        self.hessian_f = hessian_f
        self.epsilon = epsilon
        # Time that each model takes to run one iteration
        self.iteration_times = {}
        # Time that each model takes to run until convergence
        self.total_times = {}

    def error(self, x):
        return np.linalg.norm(self.grad_f(x))
```

Methodology

```python
# To find out the processing time for optimum descent
def calibrate(self, x0, lr: float = 0.01, beta: float = 0.9, alpha0: float = 1.0, c1: float = 1e-4, rho: float = 0.5,
              m: int = 5):
    initial_error = self.error(x0)

    start_gd = time.time()
    x1_gd = self.gradient_descent(x0, max_steps=1, lr=lr, beta=beta)
    end_gd = time.time()

    start_newton = time.time()
    x1_newton = self.damped_newton(x0, max_steps=1, alpha0=alpha0, c1=c1, rho=rho)
    end_newton = time.time()

    start_bfgs = time.time()
    self.bfgs(x0, max_steps=1)
    end_bfgs = time.time()

    start_l_bfgs = time.time()
    self.l_bfgs(x0, max_steps=1, m=m)
    end_l_bfgs = time.time()

    self.iteration_times["gd"] = end_gd - start_gd
    self.iteration_times["newton"] = end_newton - start_newton
    self.iteration_times["bfgs"] = end_bfgs - start_bfgs
    self.iteration_times["l_bfgs"] = end_l_bfgs - start_l_bfgs

    if(self.error(x1_gd)) != 0:
        n_iterations_gd = math.log(self.epsilon/initial_error)/math.log(self.error(x1_gd)/initial_error)
    else:
        n_iterations_gd = 1
    self.total_times["gd"] = n_iterations_gd * self.iteration_times["gd"]

    if(self.error(x1_newton)) != 0:
        c = self.error(x1_newton)/initial_error**2
        n_iterations_newton = math.log2((1+math.log(self.epsilon)/math.log(c))/(1+math.log(initial_error)/math.log(c)))
    else:
        n_iterations_newton = 1
    self.total_times["newton"] = n_iterations_newton * self.iteration_times["newton"]

    n_iterations_bfgs = math.sqrt(n_iterations_gd * n_iterations_newton)
    self.total_times["bfgs"] = n_iterations_bfgs * self.iteration_times["bfgs"]
    self.total_times["l_bfgs"] = math.sqrt(n_iterations_gd * n_iterations_bfgs) * self.iteration_times["l_bfgs"]
```

```python
def gradient_descent(self, x0, max_duration = None, max_steps = None, lr: float = 0.01, beta: float = 0.9):
    return gradient_descent(self.grad_f, x0, max_duration, max_steps, lr, beta, self.epsilon)

def damped_newton(self, x0, max_duration = None, max_steps = None, alpha0: float = 1.0, c1: float = 1e-4, rho: float = 0.5):
    return damped_newton(self.f, self.grad_f, self.hessian_f, x0, max_duration, max_steps, alpha0, c1, rho, self.epsilon)

def bfgs(self, x0, max_duration = None, max_steps = None, alpha0: float = 1.0, c1: float = 1e-4, rho: float = 0.5):
    return bfgs(self.f, self.grad_f, x0, max_duration, max_steps, alpha0, c1, rho, self.epsilon)

def l_bfgs(self, x0, max_duration = None, max_steps = None, m: int = 5, alpha0: float = 1.0, c1: float = 1e-4,
           rho: float = 0.5):
    return l_bfgs(self.f, self.grad_f, x0, max_duration, max_steps, m, alpha0, c1, rho, self.epsilon)
```

Methodology

```python
def optimum_descent(self, x0, max_time: float = 15.0, lr: float = 0.01, beta: float = 0.9, alpha0: float = 1.0,
                    c1: float = 1e-4, rho: float = 0.5, m: int = 5):
    # Hierarchy: Damped Newton, BFGS, L-BFGS, Gradient Descent
    x = x0
    start = time.time()
    if (self.total_times["newton"] <= max_time) or (self.total_times["newton"] == min(self.total_times.values())):
        print("Damped Newton is used exclusively")
        while True:
            # Check that the Hessian is positive definite
            if(np.all(np.linalg.eigvals(self.hessian_f(x)) > 0)):
                x = self.damped_newton(x, None, 1, alpha0, c1, rho)
            else:
                print("BFGS used")
                x = self.bfgs(x, None, 1, alpha0, c1, rho)
            if time.time() - start >= max_time:
                break

    elif self.total_times["bfgs"] <= max_time:
        print("BFGS and Damped Newton are used")
        while True:
            if(np.all(np.linalg.eigvals(self.hessian_f(x)) > 0)):
                x = self.damped_newton(x, None, 1, alpha0, c1, rho)
            else:
                print("BFGS used")
                x = self.bfgs(x, None, 1, alpha0, c1, rho)
            if time.time() - start >= max_time - self.total_times["bfgs"]:
                break
        x = self.bfgs(x, self.total_times["bfgs"], None, alpha0, c1, rho)
```

```python
    elif self.total_times["l_bfgs"] <= max_time:
        print("L-BFGS and Damped Newton are used")
        while True:
            if(np.all(np.linalg.eigvals(self.hessian_f(x)) > 0)):
                x = self.damped_newton(x, None, 1, alpha0, c1, rho)
            else:
                print("BFGS used")
                x = self.bfgs(x, None, 1, alpha0, c1, rho)
            if time.time() - start >= max_time - self.total_times["l_bfgs"]:
                break
        x = self.l_bfgs(x, self.total_times["l_bfgs"], None, m, alpha0, c1, rho)

    elif self.total_times["gd"] <= max_time:
        print("Gradient Descent and Damped Newton are used")
        while True:
            if(np.all(np.linalg.eigvals(self.hessian_f(x)) > 0)):
                x = self.damped_newton(x, None, 1, alpha0, c1, rho)
            else:
                print("BFGS used")
                x = self.bfgs(x, None, 1, alpha0, c1, rho)
            if time.time() - start >= max_time - self.total_times["gd"]:
                break
        x = self.gradient_descent(x, self.total_times["gd"], None, lr, beta)

    elif self.total_times["bfgs"] == min(self.total_times.values()):
        print("BFGS is used exclusively")
        x = self.bfgs(x, max_time, None, alpha0, c1, rho)
    elif self.total_times["l_bfgs"] == min(self.total_times.values()):
        print("L-BFGS is used exclusively")
        x = self.l_bfgs(x, max_time, None, m, alpha0, c1, rho)
    else:
        print("Gradient Descent is used exclusively")
        x = self.gradient_descent(x, max_time, None, lr, beta)

    return x
```

Results

Clear “preference” for Damped Newton. Equal result for all algorithms in all functions when using these settings within a very small margin of error.

- Exception 1: Calibration breakdown for certain starting points of f(x1, x2) = x1 cos(x1) + x2 sin(x2) due to a negative estimated number of iterations → Optimum algorithm does not work → Fallback to traditional algorithms.
- Exception 2: Gradient descent overflow for the Rosenbrock function.

Results

f(x1,...,xn) = x1^2 + ... + xn^2, with max time = 2 s (quadratic): Damped Newton from 0 to 6000 dimensions, L-BFGS + Damped Newton for higher dimensions.

f(x1,...,xn) = x1exp(x1) + ... + xnexp(xn), with max time = 2 s: Damped Newton from 0 to 3000 dimensions, L-BFGS + Damped Newton from 3000 to 12000 dimensions, Gradient Descent for higher dimensions.

Conclusion

- Robust model useful for both optimization and comparisons between optimization algorithms.
- Can be modified easily for machine learning tasks by optimizing a loss function.
- Confirms that Newton is superior for quadratic optimization up to a relatively high dimensionality.
- Weaknesses: assumptions made that may be unrealistic.
    - The time per iteration tends to be similar.
    - The approximation for the Newton and Gradient Descent iterations is close to the real number.
    - BFGS and L-BFGS geometric mean approximation. Alternative: obtaining an equation from several iterations. Problems: unfeasible for smaller problems, costly.
    - Positive values in the estimated number of iterations.

Code

https://drive.google.com/file/d/1khtUOPNyk7IXx5HRTj__jWGAf0OpZm__/view?usp=sharing